

Tarea 7

Nombre y apellido: Nicolás Seguel Número de lista: 49 Puntaje total: 21

El objetivo de esta tarea es comprender en la práctica la relación entre los dominios del espacio y de la frecuencia. Para eso trabajarán con la convolución, transformada de Fourier y transformada de Radon (teorema de la sección central).

- 6
1. [6p] Convolución lineal versus cíclica.

1

 - a. [1p] Defina dos o más imágenes, $f_i[m, n]$, una simple que tenga una definición analítica (con las funciones vistas en clases) y otra más realista, que puede ser una fotografía. Defina también dos o más filtros, $h_i[m, n]$, por ejemplo, uno que suavice, que encuentre los gradientes de la imagen, o algún otro que sea interesante.

3

 - b. [3p] Calcule y visualice la convolución lineal y cíclica (sin acolchamiento y con acolchamiento) de las imágenes con los filtros. Discuta el tamaño del acolchamiento necesario para que la convolución cíclica entregue el mismo resultado que la lineal.

2

 - c. [2p] Pruebe estas convoluciones con imágenes en color, aplicando el filtro a cada uno de los canales. Los canales típicos son RGB, pero también pruebe HSV y YCbCr ¿Hace alguna diferencia a qué canal aplicar el filtro?

6

 2. [6p] Costo de convolucionar directamente en el espacio versus la frecuencia.

4

 - a. [4p] Para diferentes tamaños de kernel $h[m, n]$, $N \times M$, mida el tiempo de ejecución de:
 - La convolución directa en el dominio espacial.
 - La convolución implementada como multiplicación en el dominio de la frecuencia:

$$g = 2\text{DFT}^{-1}\{2\text{DFT}\{f\} 2\text{DFT}\{h\}\}.$$

1

 - b. [1p] Grafique el tiempo en función del tamaño del kernel y determine el punto de cruce en que la convolución en frecuencia se vuelve más eficiente.

1

 - c. [1p] Explique conceptualmente por qué el método en frecuencia es más ventajoso para kernels grandes, y qué rol cumple el acolchamiento en este cálculo.

7

 3. [7p] El teorema de la sección central establece que la transformada de Fourier de la proyección de una función 2D (su transformada de Radon) corresponde a una sección central de la transformada 2D de la función.

- 3 a. [3p] Genere una imagen 2D $f(x, y)$ analítica (por ejemplo, suma de rectángulos o gaussiana anisotrópica) y calcule numéricamente
- la transformada de Fourier 2D de la imagen, y
 - la transformada de Radon $g_\theta(R)$ para algunos ángulos θ .
- 2 b. [2p] Para cada ángulo θ , tome la transformada de Fourier 1D de la proyección $g_\theta(R)$ y compárela con la sección central de la transformada 2D en dirección θ .
- 2 c. [2p] Visualice ambos resultados superpuestos y discuta la concordancia observada.
- 2 4. [6p] Abel-Fourier-Hankel. Se define la función “copa” como

$$f(r) = \frac{2}{\sqrt{1-4R^2}} \cap(r).$$

y la función cono como $g(r) = \wedge(r)$

- 0 a. [2p] Encuentre analíticamente el ciclo de Abel-Fourier-Hankel para la copa: $f(r)$.
- 2 b. [2p] Muestre gráficamente las funciones del ciclo para la copa.
- 0 c. [2p] Ahora encuentre numéricamente el ciclo de Abel-Fourier-Hankel para
- la función copa (misma del punto anterior), y
 - la función cono $g(r)$.

IEE3584 - Procesamiento Avanzado de Señales

Tarea 7

Nicolás Seguel

Septiembre 2025

1. Convolución lineal vs cíclica

a) Imágenes a utilizar

Se utilizará una imagen real y una imagen binaria simple, que corresponde a un círculo blanco. Los filtros a utilizar son un filtro pasa bajos ideal, es decir, un cuadrado de 25×25 con valores iguales a $\frac{1}{25^2}$, y un filtro de Sobel para detección de bordes.

b) Convolución lineal y cíclica

Se aplica la convolución lineal y cíclica a la imagen en escala de grises, utilizando ambos filtros.

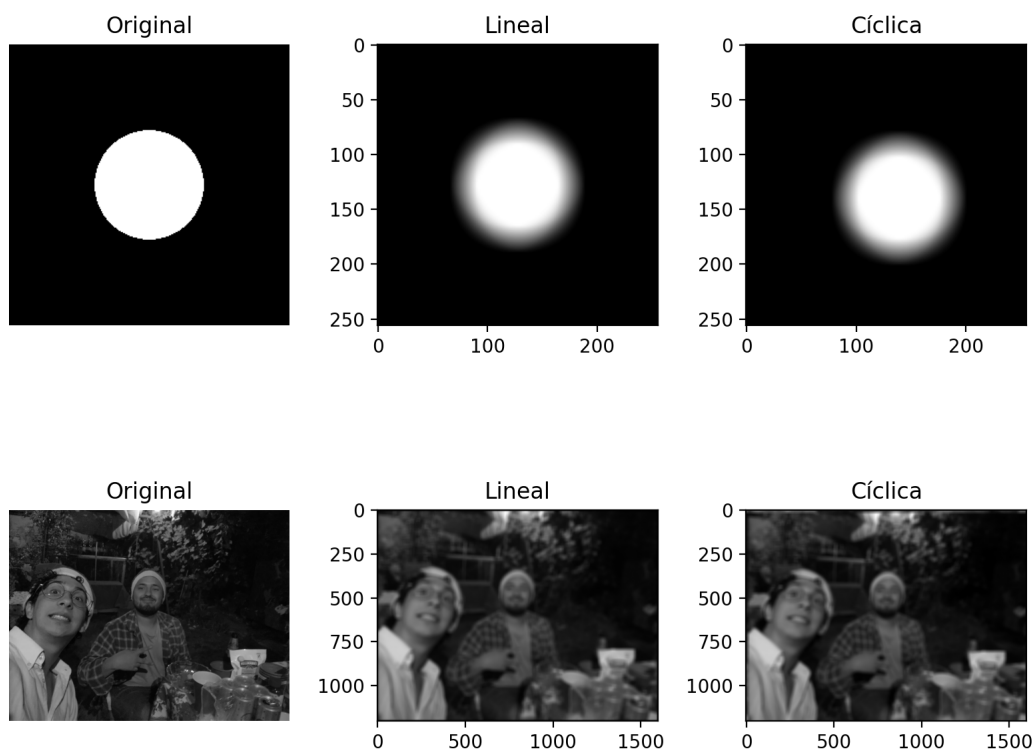


Figura 1: Resultados de la convolución lineal y cíclica con un filtro pasa bajos.



Figura 3: Resultados de la convolución cíclica en espacios RGB, HSV y YCbCr.

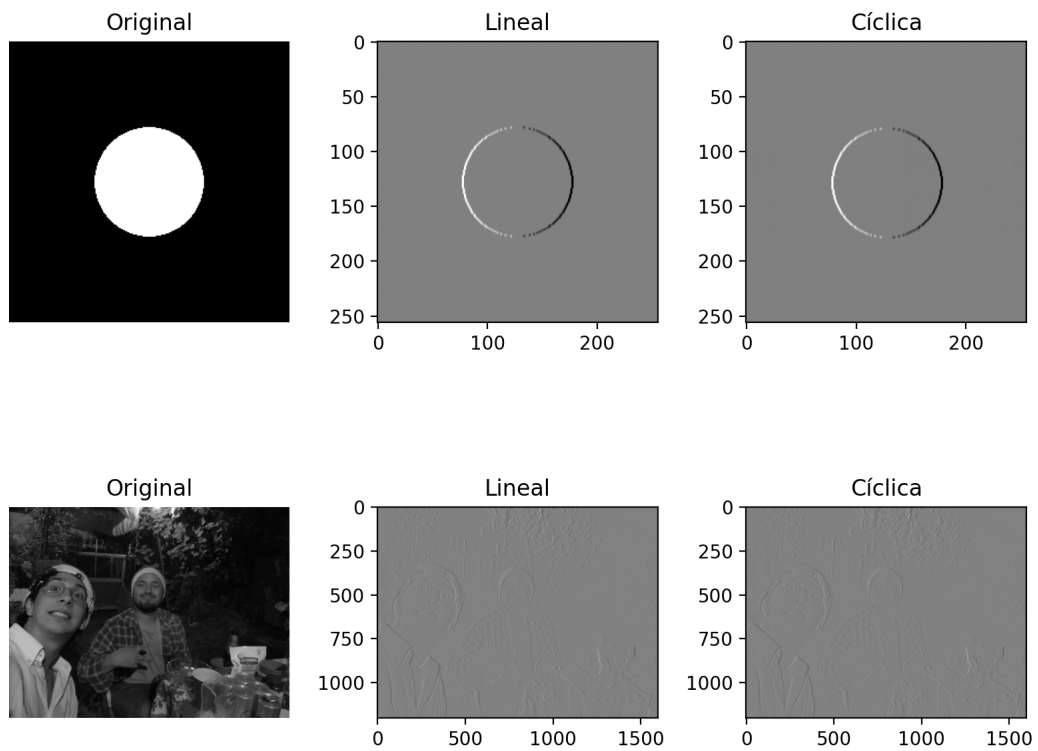


Figura 2: Resultados de la convolución lineal y cíclica con un filtro de Sobel.

Se puede ver que la convolución cíclica introduce un efecto de borde en las imágenes. Mientras más grande el tamaño de la máscara, más evidente es este efecto. Si se desea evitar este efecto en la convolución cíclica, se puede aplicar un padding a la imagen original. Esto se debe a que el padding agrega un borde adicional que reduce la interferencia entre los bordes de la imagen original y las copias generadas por la naturaleza de la convolución cíclica.

c) Convoluciones en canales RGB, HSV y YCbCr

La convolución en el espacio HSV afecta los colores de la imagen, ya que el canal de saturación y valor se ven modificados.

```
1 # p1_convolucion_lineal_vs_ciclica.py
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.signal import convolve2d
5 from skimage import io, color, img_as_float
6 import os
7
8 os.makedirs("figs", exist_ok=True)
9
10 # a) Imagenes a utilizar y filtros
11
12 def circ(N=256, R=60):
13     """
14     Genera una imagen circular (circ function) de tamaño NxN y radio R.
15     """
16     x = np.linspace(-N/2, N/2, N)
17     y = np.linspace(-N/2, N/2, N)
18     X, Y = np.meshgrid(x, y)
19     r = np.sqrt(X**2 + Y**2)
20     img = np.where(r <= R, 1.0, 0.0)
21     return img
22
23 # Imagen analítica circular
24 img_simple = circ(N=256, R=50)
25
26 # Imagen real
27 img_real = img_as_float(io.imread("figs/feliz_navidad.jpg"))
28 img_gray = color.rgb2gray(img_real)
29
30 # Filtros
31 filtro_suavizado = np.ones((25, 25)) / 25**2
32 filtro_sobel = np.array([[1, 0, -1],
33                           [2, 0, -2],
34                           [1, 0, -1]])
35
36 # b) Convolución lineal vs cíclica
37 def conv_lineal(f, h):
38     return convolve2d(f, h, mode='same', boundary='fill', fillvalue=0)
39
40 def conv_ciclica(f, h):
41     F = np.fft.fft2(f)
42     H = np.fft.fft2(h, s=f.shape)
43     return np.real(np.fft.ifft2(F * H))
44
45 # Ejemplo con imagen simple
46 cl_simple_suave = conv_lineal(img_simple, filtro_suavizado)
47 cc_simple_suave = conv_ciclica(img_simple, filtro_suavizado)
48
49 cl_simple_sobel = conv_lineal(img_simple, filtro_sobel)
50 cc_simple_sobel = conv_ciclica(img_simple, filtro_sobel)
51
52 cl_gray_suave = conv_lineal(img_gray, filtro_suavizado)
53 cc_gray_suave = conv_ciclica(img_gray, filtro_suavizado)
54
```

```

55 cl_gray_sobel = conv_lineal(img_gray, filtro_sobel)
56 cc_gray_sobel = conv_lineal(img_gray, filtro_sobel)
57
58 plt.figure(figsize=(8, 7))
59 plt.subplot(2, 3, 1); plt.imshow(img_simple, cmap='gray'); plt.title("Original");
    plt.axis('off')
60 plt.subplot(2, 3, 2); plt.imshow(cl_simple_suave, cmap='gray'); plt.title("Lineal")
61 plt.subplot(2, 3, 3); plt.imshow(cc_simple_suave, cmap='gray'); plt.title("Cíclica"
    )
62 plt.subplot(2, 3, 4); plt.imshow(img_gray, cmap='gray'); plt.title("Original");plt.
    axis('off')
63 plt.subplot(2, 3, 5); plt.imshow(cl_gray_suave, cmap='gray'); plt.title("Lineal")
64 plt.subplot(2, 3, 6); plt.imshow(cc_gray_suave, cmap='gray'); plt.title("Cíclica")
65 plt.tight_layout()
66 plt.savefig("figs/p1_convolucion_suave.png", dpi=200)
67
68 plt.figure(figsize=(8, 7))
69 plt.subplot(2, 3, 1); plt.imshow(img_simple, cmap='gray'); plt.title("Original");
    plt.axis('off')
70 plt.subplot(2, 3, 2); plt.imshow(cl_simple_sobel, cmap='gray'); plt.title("Lineal")
71 plt.subplot(2, 3, 3); plt.imshow(cc_simple_sobel, cmap='gray'); plt.title("Cíclica"
    )
72 plt.subplot(2, 3, 4); plt.imshow(img_gray, cmap='gray'); plt.title("Original");plt.
    axis('off')
73 plt.subplot(2, 3, 5); plt.imshow(cl_gray_sobel, cmap='gray'); plt.title("Lineal")
74 plt.subplot(2, 3, 6); plt.imshow(cc_gray_sobel, cmap='gray'); plt.title("Cíclica")
75 plt.tight_layout()
76 plt.savefig("figs/p1_convolucion_sobel.png", dpi=200)
77
78 # c) Aplicar a canales RGB y HSV
79 img_hsv = color.rgb2hsv(img_real)
80 img_ycbcr = color.rgb2ycbcr(img_real)
81 for espacio, datos in zip(['RGB', 'HSV', 'YCbCr'], [img_real, img_hsv, img_ycbcr]):
82     filtrada = np.zeros_like(datos)
83     for i in range(3):
84         filtrada[:, :, i] = conv_ciclica(datos[:, :, i], filtro_suavizado)
85     if espacio == 'HSV':
86         filtrada = color.hsv2rgb(filtrada)
87     if espacio == 'YCbCr':
88         filtrada = color.ycbcr2rgb(filtrada)
89 plt.imsave(f"figs/p1_convolucion_{espacio.lower()}.png", filtrada)

```

Listing 1: p1_convolucion_lineal_vs_ciclica.py

2. Costo computacional de la convolución

a) Convolución espacial y frecuencial

Para distintos tamaños del kernel (de 3×3 a 255×255), se mide el tiempo de ejecución con `time.time()`.

b) Tiempos de ejecución

El gráfico anterior muestra que la convolución en el dominio espacial es más eficiente para tamaños de kernel pequeños, mientras que la convolución en el dominio frecuencial se vuelve más

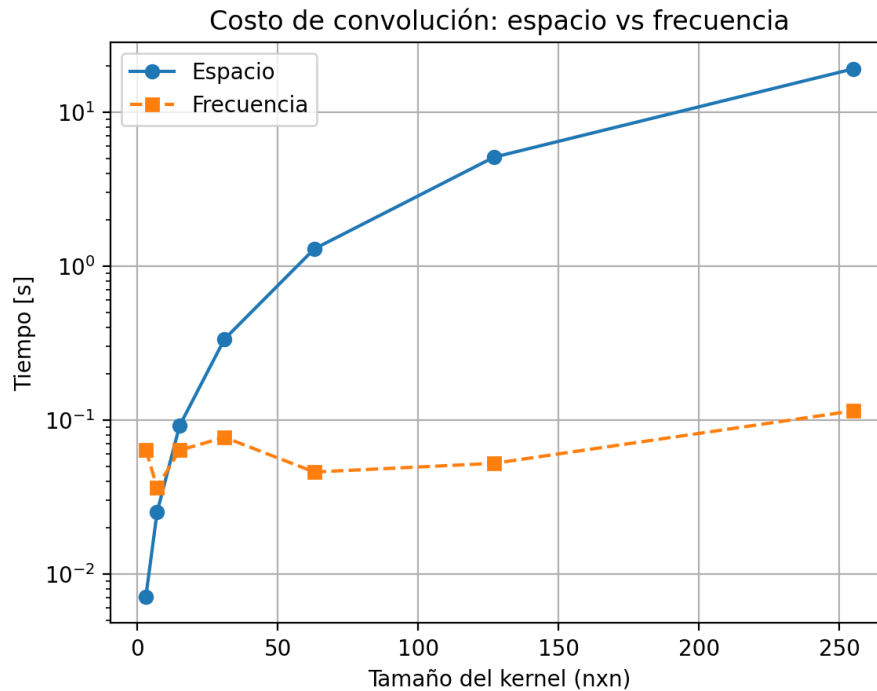


Figura 4: Costo computacional de la convolución espacial y frecuencial en función del tamaño del filtro.

eficiente a medida que el tamaño del kernel aumenta. El punto de cruce entre ambas curvas está cerca de un tamaño de kernel de 10×10 .

c) Ventaja de la convolución en frecuencia

La convolución en espacio calcula el producto punto entre el kernel y la región de la imagen para cada píxel, lo que implica un costo computacional que crece con el tamaño del kernel. En contraste, la convolución en frecuencia utiliza la Transformada Rápida de Fourier (FFT) para transformar tanto la imagen como el kernel al dominio frecuencial, donde la convolución se realiza mediante una multiplicación punto a punto. La FFT tiene un costo computacional de $O(N \log N)$, lo que la hace más eficiente para kernels grandes.

```

1 # p2_costo_convolucion.py
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.signal import convolve2d
5 import time
6 import os
7
8 os.makedirs("figs", exist_ok=True)
9
10 img = np.random.rand(512, 512)
11
12 def conv_espacial(img, h):
13     return convolve2d(img, h, mode='same')
14
15 def conv_freq(img, h):

```



```

16     pad_shape = [img.shape[0] + h.shape[0] - 1, img.shape[1] + h.shape[1] - 1]
17     F = np.fft.fft2(img, pad_shape)
18     H = np.fft.fft2(h, pad_shape)
19     conv = np.real(np.fft.ifft2(F * H))
20     return conv[:img.shape[0], :img.shape[1]]
21
22 sizes = [3, 7, 15, 31, 63, 127, 255]
23 t_espacial, t_freq = [], []
24
25 for n in sizes:
26     h = np.ones((n, n)) / n**2
27     t0 = time.time(); conv_espacial(img, h); t1 = time.time()
28     t_espacial.append(t1 - t0)
29     t0 = time.time(); conv_freq(img, h); t1 = time.time()
30     t_freq.append(t1 - t0)
31
32 plt.figure()
33 plt.semilogy(sizes, t_espacial, 'o-', label='Espacio')
34 plt.semilogy(sizes, t_freq, 's--', label='Frecuencia')
35 plt.xlabel("Tamaño del kernel (nxn)")
36 plt.ylabel("Tiempo [s]")
37 plt.legend()
38 plt.grid(True)
39 plt.title("Costo de convolución: espacio vs frecuencia")
40 plt.savefig("figs/p2_tiempos_convolucion.png", dpi=200)

```

Listing 2: p2_costo_convolucion.py

3. Teorema de la Sección Central

a) Transformada de Fourier y Radon

Se genera una imagen analítica formada por un rectángulo, sobre la cual se calcula:

- La transformada de Fourier 2D.
- La transformada de Radon en distintos ángulos.

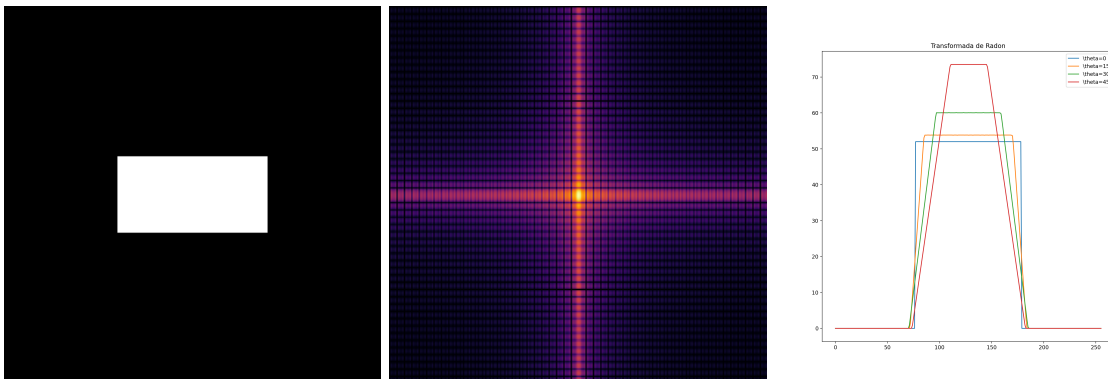


Figura 5: Imagen base, transformada de Fourier 2D y transformada de Radon.

b) Comparación de secciones

Se compara la transformada de Fourier de las proyecciones de Radon con las secciones centrales de la transformada 2D, verificando el teorema de la sección central.

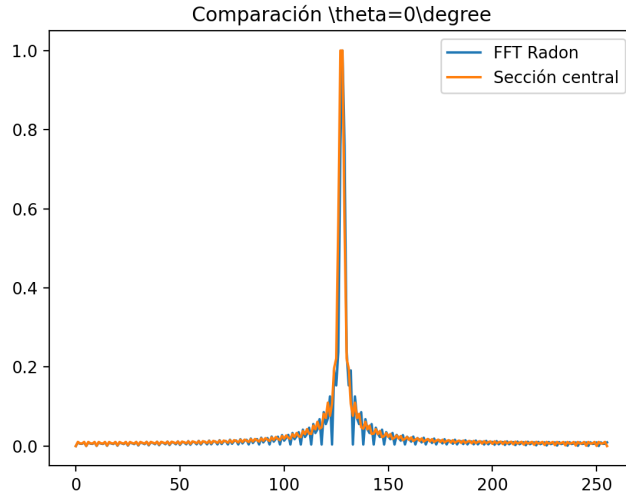


Figura 6: Ejemplo de comparación entre la FFT de Radon y la sección central del espectro.

c) Resultados

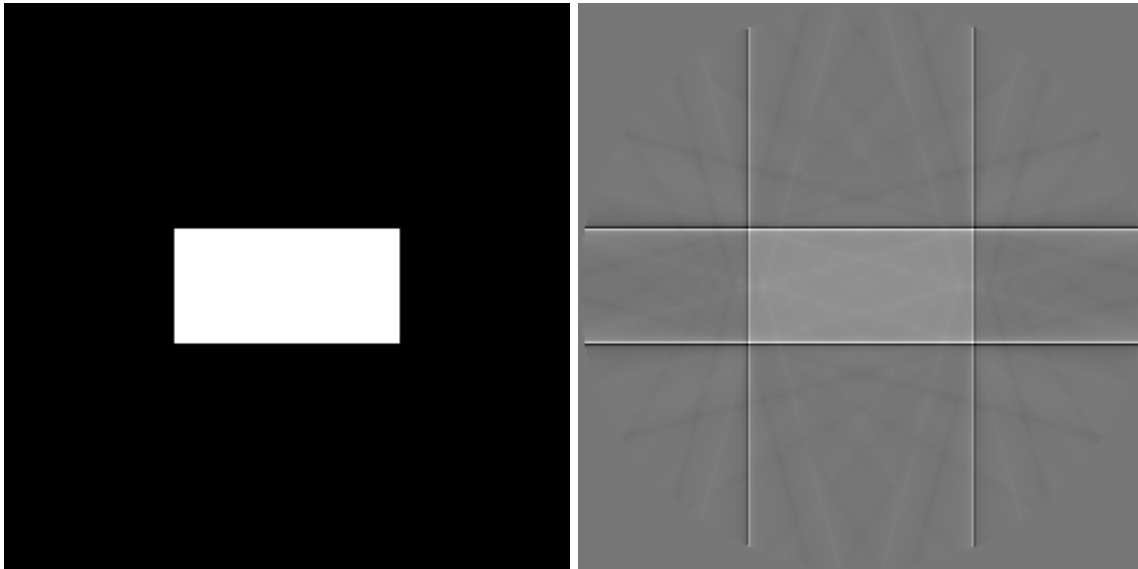


Figura 7: Imagen base y reconstrucción mediante retroproyección filtrada.

La reconstrucción mediante retroproyección filtrada a partir de las proyecciones de Radon se asemeja bastante a la imagen original. Sin embargo, se observa líneas que se extienden a lo largo de los bordes del bloque, además de una difuminación en el color de la imagen. Esto se debe a la falta de ángulos en la transformada de Radon, lo que genera artefactos en la reconstrucción.

```

1 # p3_teorema_seccion_central.py
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from skimage.transform import radon, iradon
5 import scipy.ndimage as nd
6 import os
7
8 os.makedirs("figs", exist_ok=True)
9
10 # a) Imagen analítica: suma de rectángulos
11 x = np.linspace(-1, 1, 256)
12 y = np.linspace(-1, 1, 256)
13 X, Y = np.meshgrid(x, y)
14 img = ((np.abs(X) < 0.4) & (np.abs(Y) < 0.2)).astype(float)
15 plt.imsave("figs/p3_imagen_base.png", img, cmap='gray')
16
17 # Transformada de Fourier 2D
18 F2 = np.fft.fftshift(np.abs(np.fft.fft2(img)))
19 plt.imsave("figs/p3_fourier2d.png", np.log1p(F2), cmap='inferno')
20
21 # Transformada de Radon
22 thetas = np.linspace(0, 180, 12, endpoint=False)
23 R = radon(img, theta=thetas, circle=True)
24 plt.figure(figsize=(10,10))
25 plt.plot(R[:, 0], label=f'\\theta={thetas[0]:.0f}')
26 plt.plot(R[:, 1], label=f'\\theta={thetas[1]:.0f}')
27 plt.plot(R[:, 2], label=f'\\theta={thetas[2]:.0f}')
28 plt.plot(R[:, 3], label=f'\\theta={thetas[3]:.0f}')
29 plt.legend(); plt.title("Transformada de Radon")
30 plt.savefig("figs/p3_radon.png", dpi=200)
31
32 # b) Comparar secciones centrales
33
34 for i, theta in enumerate(thetas):
35     g = R[:, i]
36     G = np.abs(np.fft.fftshift(np.fft.fft(g)))
37     N = F2.shape[0]
38     x = np.arange(-N//2, N//2)
39     X, Y = np.meshgrid(x, x)
40
41     # Coordenadas de línea (por interpolación)
42     r_line = np.linspace(-N//2, N//2, N)
43     x_line = N/2 + r_line * np.cos(np.deg2rad(theta))
44     y_line = N/2 + r_line * np.sin(np.deg2rad(theta))
45
46     F_section = nd.map_coordinates(F2, [y_line, x_line], order=1)
47     F_section /= F_section.max()
48
49     plt.figure()
50     plt.plot(G / G.max(), label='FFT Radon')
51     plt.plot(F_section, label='Sección central')
52     plt.legend()
53     plt.title(f"Comparación \\theta={theta:.0f}\\degree")
54     plt.savefig(f"figs/p3_comparacion_theta{int(theta)}.png", dpi=200)
55
56 # ----- Reconstrucción con iradon (FBP) -----
57 recon_iradon = iradon(R, theta=thetas, circle=True)

```

```
58 plt.imsave("figs/p3c_recon_iradon.png", recon_iradon, cmap='gray')
```

Listing 3: p3_teorema_seccion_central.py

4. Ciclo Abel-Fourier-Hankel

b) Gráficos de la transformada

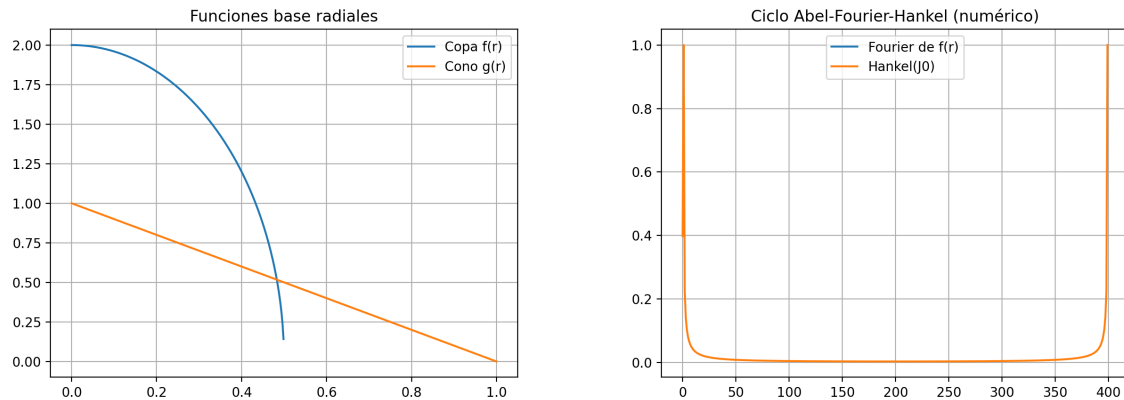


Figura 8: Funciones base radiales y comparación de sus transformadas en el ciclo AFH.

```
1 # p4_abel_fourier_hankel.py
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.special import j0
5 import os
6
7 os.makedirs("figs", exist_ok=True)
8
9 # Definición de funciones radiales
10 r = np.linspace(0, 1, 400)
11 cup = 2 * np.sqrt(1 - 4 * r**2) * (np.abs(r) <= 0.5)
12 cone = np.maximum(0, 1 - r)
13
14 # Abel-Fourier-Hankel ciclo (conceptual)
15 A = np.cumsum(cup[::-1])[::-1]
16 F = np.fft.fftshift(np.abs(np.fft.fft(cup)))
17 H = np.abs(np.fft.fft(j0(2*np.pi*r)))
18
19 plt.figure()
20 plt.plot(r, cup, label='Copa f(r)')
21 plt.plot(r, cone, label='Cono g(r)')
22 plt.title("Funciones base radiales")
23 plt.legend(); plt.grid(True)
24 plt.savefig("figs/p4_funciones_base.png", dpi=200)
25
26 plt.figure()
27 plt.plot(F / F.max(), label='Fourier de f(r)')
28 plt.plot(H / H.max(), label='Hankel(J0)')
29 plt.legend(); plt.title("Ciclo Abel-Fourier-Hankel (numérico)")
```

```
30 plt.grid(True)
31 plt.savefig("figs/p4_ciclo_afh.png", dpi=200)
```

Listing 4: p4_abel_fourier_hankel.py